



# ARA0110 PADRÕES DE PROJETO

Aula 04: Padrões de projeto estruturais - parte 1





RECAPITULANDO





## PADRÕES DE PROJETOS

O grupo GoF definiu em seu livro uma classificação para agrupar padrões de projeto de acordo com sua finalidade. São elas:

- Padrões criacionais
- Padrões estruturais
- Padrões comportamentais



## PADRÕES ESTRUTURAIS

Os padrões de projeto estruturais focam na forma como a estrutura dos objetos são compostas, para formar estruturas maiores e completas.

Nesses padrões, encontramos diversas abordagens de estruturação e composição de objetos.



ADAPTER





## DEFINIÇÃO

Converter a interface de uma classe em outra interface, esperada pelos clientes. O Adapter permite que classes com interfaces incompatíveis trabalhem em conjunto - o que, de outra forma, seria impossível.

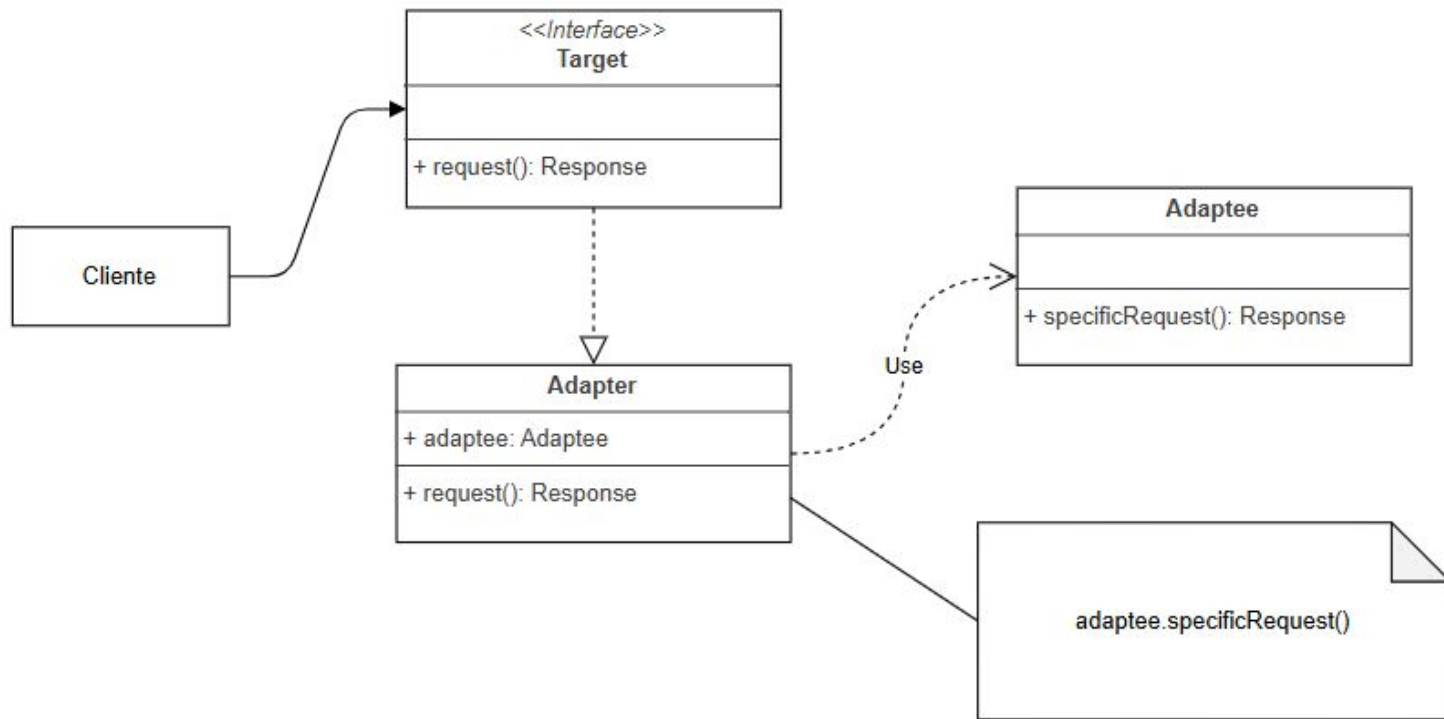
GoF, Padrões de Projetos. p.140



## MOTIVAÇÃO

Você tem um sistema que espera receber dados em JSON. No entanto, você está integrando um sistema que retorna os dados em XML. Como você não pode modificar o código do fornecedor do sistema integrado, você precisa pensar em uma forma de trabalhar com os dados em XML, mas oferecendo os dados em JSON que o seu sistema espera.

# DIAGRAMA







## PARTICIPANTES

- **Target:** interface que declara o domínio utilizado pelo sistema
- **Adaptee:** operação que deve ser adaptada ao domínio do sistema
- **Adapter:** implementação que adapta a operação integrada ao domínio do sistema
- **Cliente:** elemento que faz uso do adaptador



## UTILIZAÇÃO

A operação integrada fica encapsulada em um adaptador do sistema, que fica responsável por utilizar a operação e transforma o resultado para o domínio do sistema.

No cliente, somente o adaptador é utilizado, sem exposição da integração.

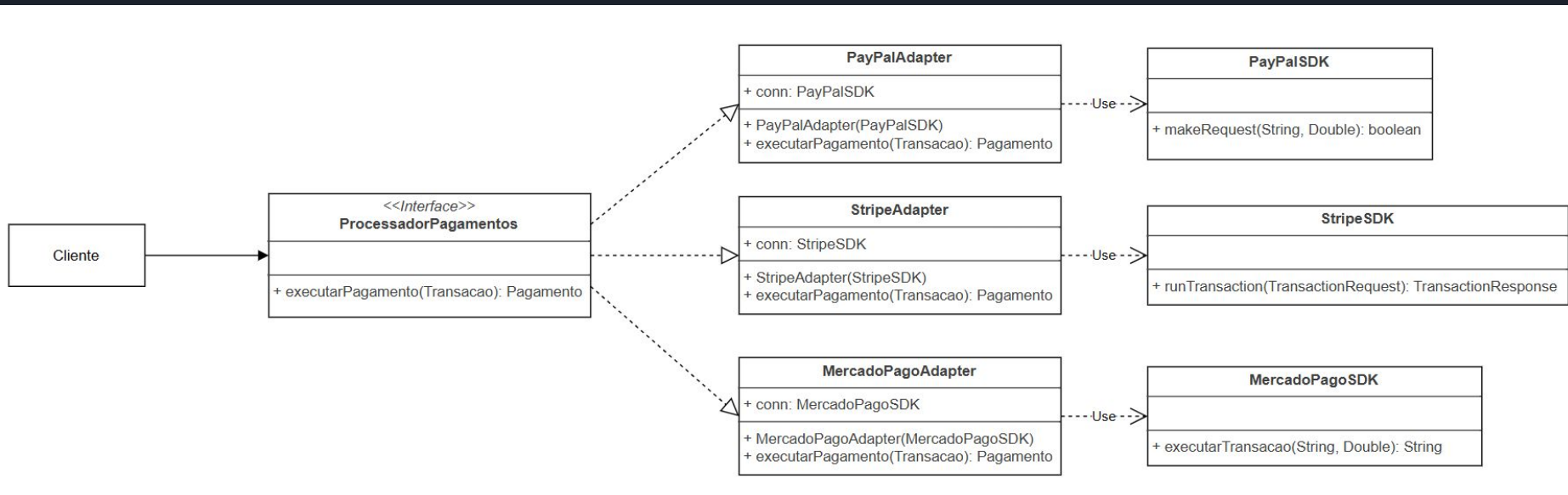


## HORA DE EXERCÍCIO

Sua loja virtual utiliza o PayPal como gateway de pagamentos. Um belo dia, você começa a integrar também o Stripe. Cada um desses gateways expõe um método específico para realizar uma transação.

Devemos criar um adaptador para integrar diferentes gateways para uma interface comum de transações da própria loja virtual.

# DIAGRAMA





## HORA DE EXERCÍCIO

Em um sistema de monitoramento de temperatura, você pode usar sensores de diferentes fabricantes, com diferentes formas de uso. Um sensor retorna a temperatura em escala **Celsius** no formato de **string** e outro retorna a temperatura em escala **Fahrenheit** no formato de **ponto flutuante**. Você deve padronizar a medição de temperatura para retornar sempre na escala **Celsius** em formato de ponto **flutuante**.

**DIAGRAMA?**



## APLICABILIDADE

- Usar uma classe já existente, mas cuja assinatura não corresponde com o sistema atual
- Um sistema que implementa uma classe reutilizável que coopere com classes não relacionadas ou não previstas
- Usar várias subclasses, mas que é impraticável adaptar suas interfaces



## CONSEQUÊNCIAS

- Adaptar classes de assinaturas distintas, favorecendo transparência
- Encapsular operações com formatos distintos
- Manter um ponto comum de utilização no sistema





BRIDGE





## DEFINIÇÃO

Desacoplar uma abstração da sua implementação, de modo que as duas possam variar independentemente.

GoF, Padrões de Projetos. p.151

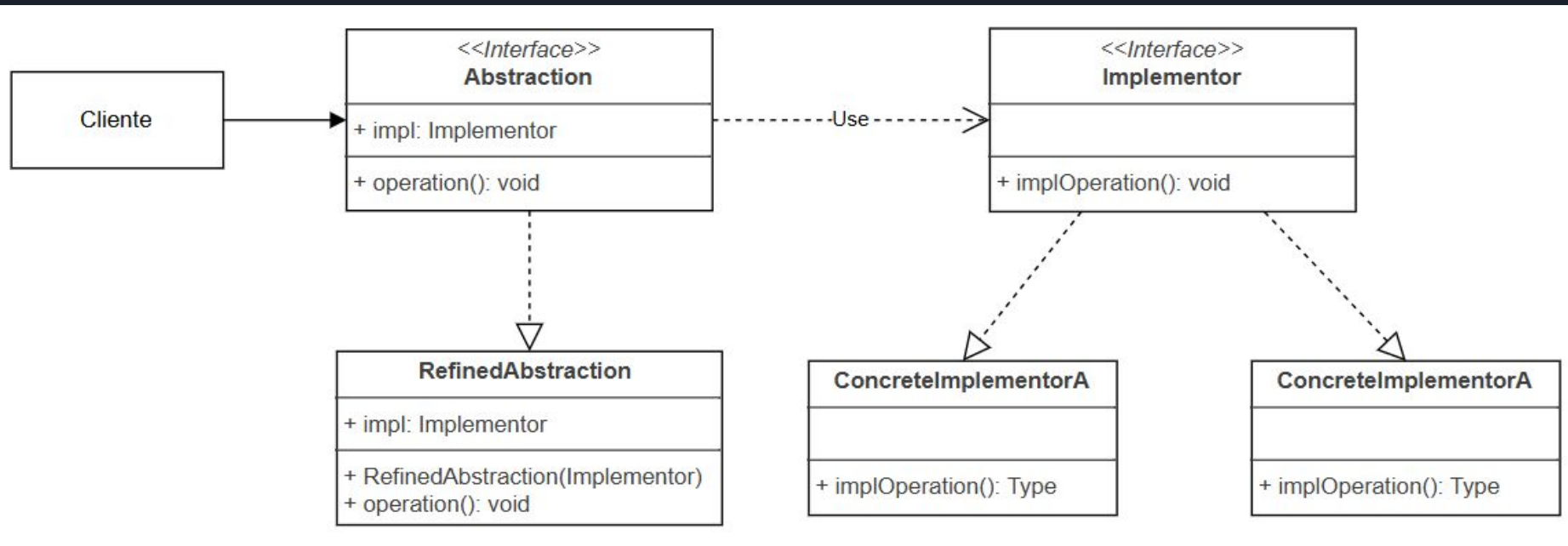
# MOTIVAÇÃO

Em um sistema de desenho gráfico, temos nossa classe **Forma** usada para criar uma forma geométrica. Nessa nossa forma, gostaríamos de aplicar cores diferentes a ela. Como temos diferentes formas e diferentes cores, seguindo uma abordagem de herança teríamos uma propagação gigantesca de sub-classes dentro do sistema.

E se, ao invés de tratarmos essas diferenças como tipos, tratássemos como composições?



# DIAGRAMA





## PARTICIPANTES

- **Abstraction:** interface que define a abstração da operação
- **RefinedAbstraction:** implementação da operação. Neste ponto, mantém-se uma referência para Implementor
- **Implementor:** interface que define operações a serem implementadas pelas classes, sem ter correspondência direta com Abstraction



## PARTICIPANTES

- **ConcreteImplementorX:**  
implementação concreta da operação definida por Implementor, que fará parte da composição
- **Cliente:** elemento que faz uso das operações de Abstraction e Implementor a partir da implementação refinada



## UTILIZAÇÃO

No cliente criamos o objeto que implementa a operação de Implementor. Após isso, conseguimos criar nossa abstração passando o objeto de Implementor para compôr a operação. Quando utilizamos a abstração, as duas operações se conectam, e podemos mudar as abstrações compostas por diferentes implementações.



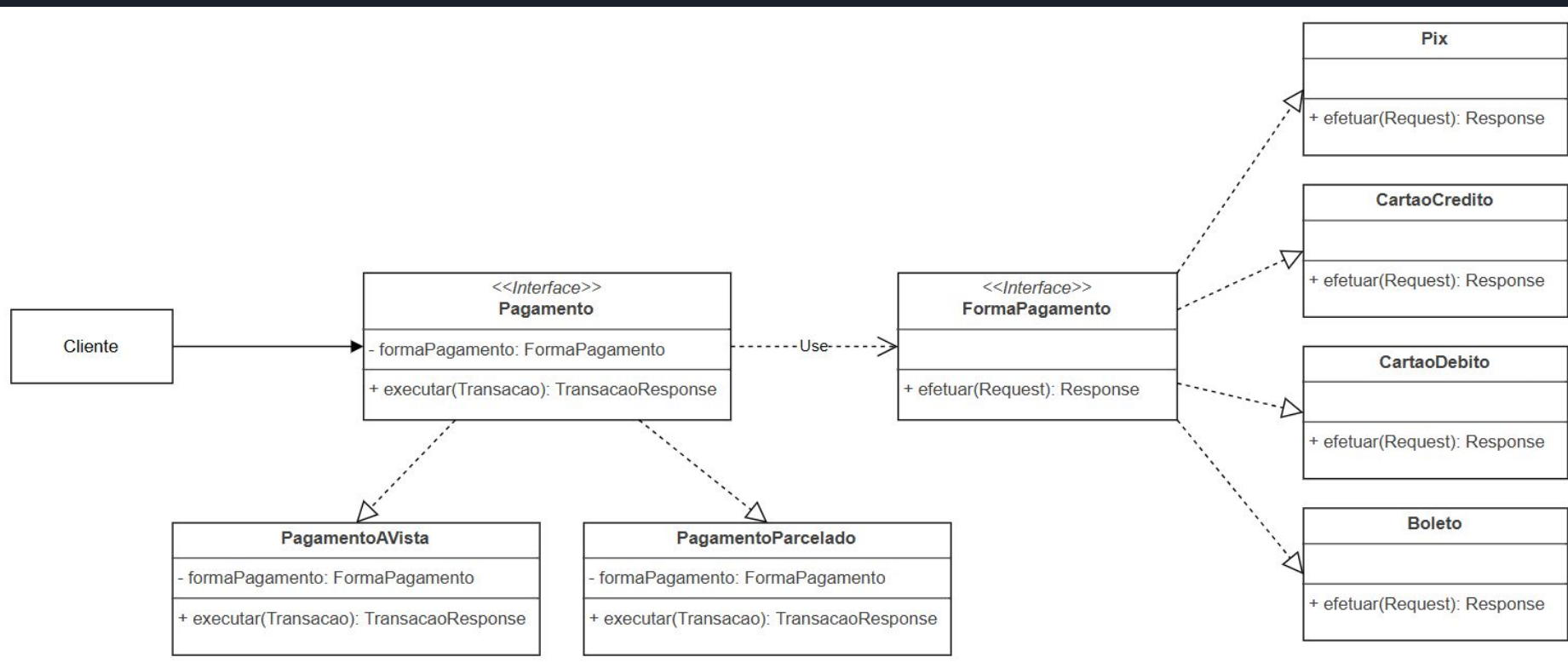
## HORA DE EXERCÍCIO

Retomando nossa loja virtual, vamos tratar das formas de pagamento previstas para compras. Como modalidades de pagamento, podemos oferecer pagamento a vista ou parcelado. Já as formas de pagamento disponíveis são PIX, cartão de crédito e débito e boleto bancário.

Vamos implementar isso no nosso sistema.



# DIAGRAMA





## HORA DE EXERCÍCIO

Vamos implementar um sistema de troca de mensagens, onde podemos enviar mensagens de diferentes tipos de prioridade entregue em vários canais de comunicação possíveis.

Devemos ser capazes de escolher o canal de comunicação no momento em que estamos configurando a mensagem a ser enviada.

**DIAGRAMA?**



## APLICABILIDADE

- Deve se evitar vínculo permanente entre uma abstração e uma implementação.
- Precisamos que as abstrações e implementações precisarem ser extensíveis
- Quisermos evitar proliferação de sub-classes



## CONSEQUÊNCIAS

- Desacoplar interface da implementação
- Extensibilidade melhorada
- Ocultação de detalhes de implementação dos clientes



COMPOSITE

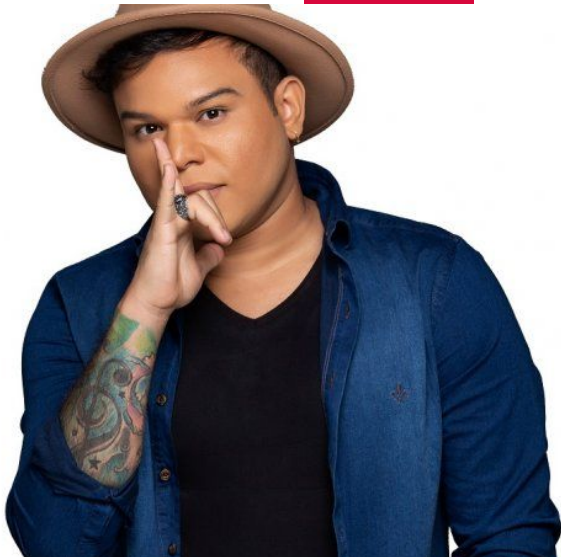




## DEFINIÇÃO

Compor objetos em estruturas de árvore para representar hierarquias partes-todo. Composite permite aos clientes tratarem de maneira uniforme objetos individuais e composições de objetos.

GoF, Padrões de Projetos. p.160

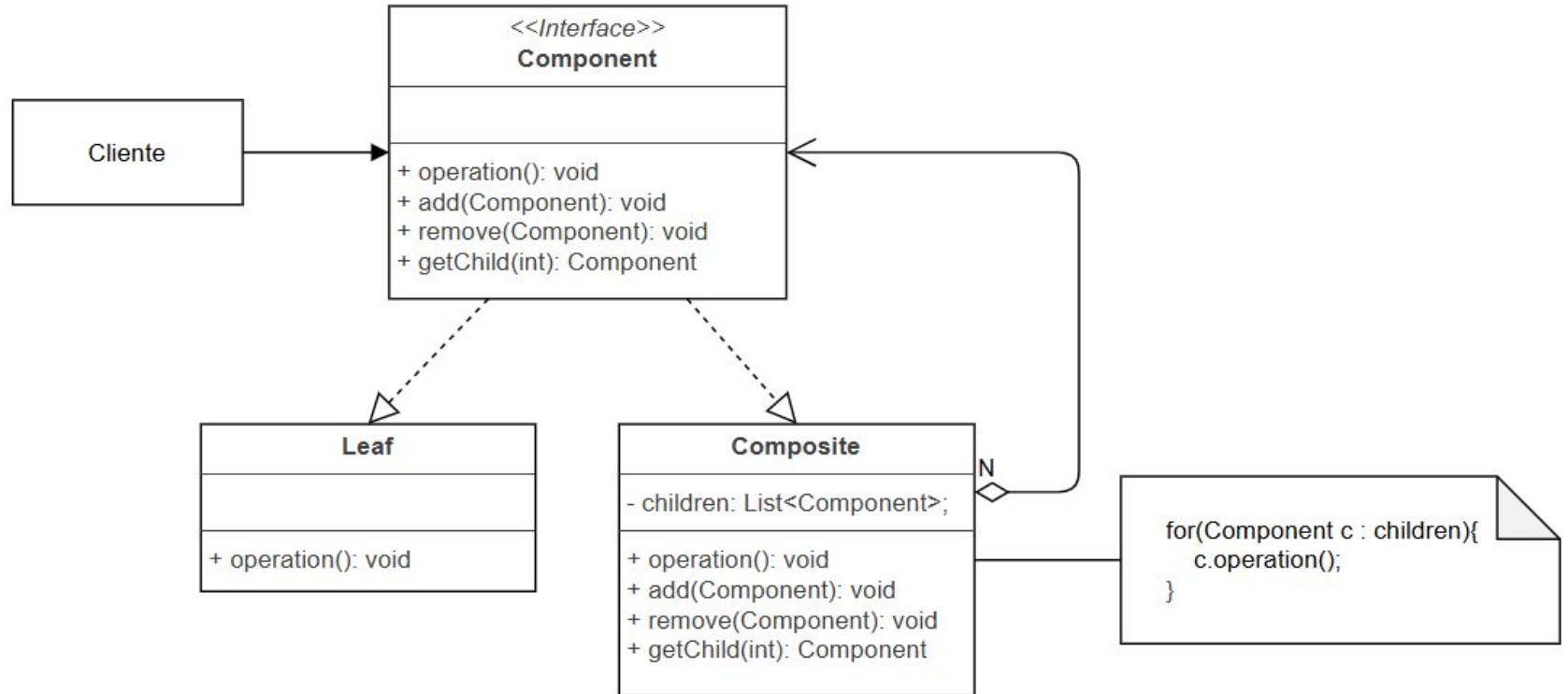


## MOTIVAÇÃO

Temos um sistema de arquivos em um sistema operacional, onde temos arquivos (folhas) e diretórios (compostos). Em um diretório temos arquivos e também podemos ter outros diretórios salvos. Para calcular o tamanho total de uma página, você não quer se preocupar com o conteúdo do diretório (arquivo ou outro diretório), você apenas pede o tamanho e recebe esta informação.



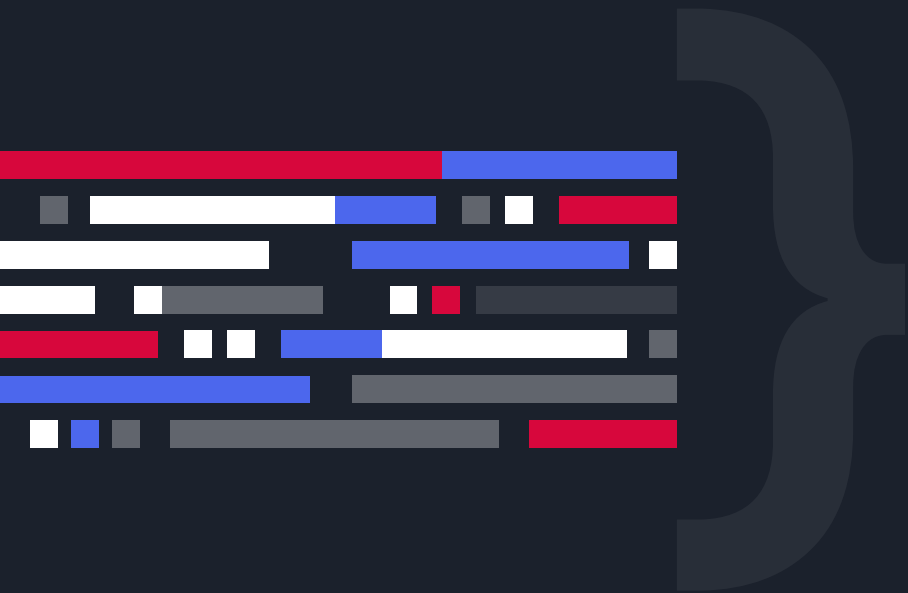
# DIAGRAMA





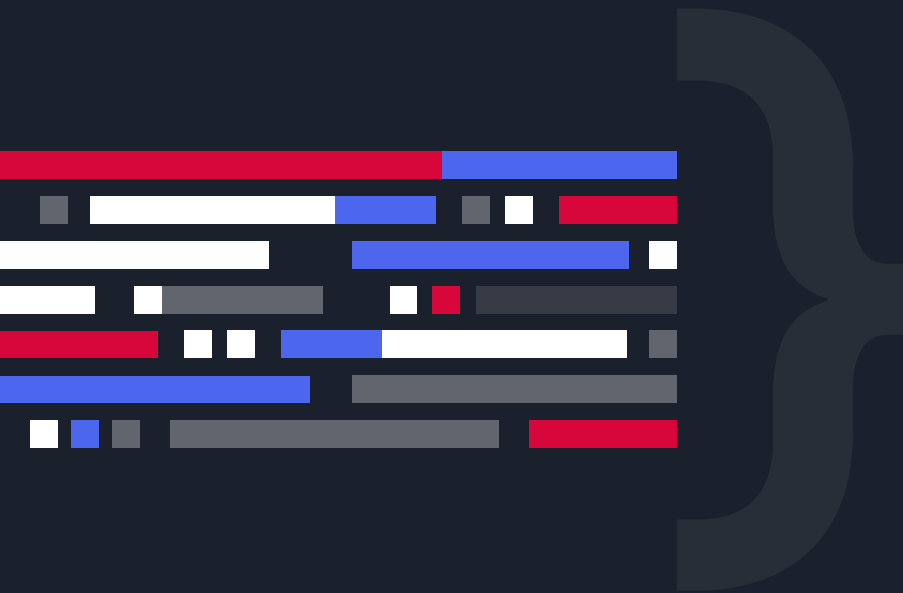
## PARTICIPANTES

- **Component:** interface que define uma operação executada, e ações para manipulação da composição
- **Leaf:** implementação da operação executada, sem a previsão de composição (folha)
- **Composite:** implementação da operação executada e das ações de manipulação da composição (compostos)



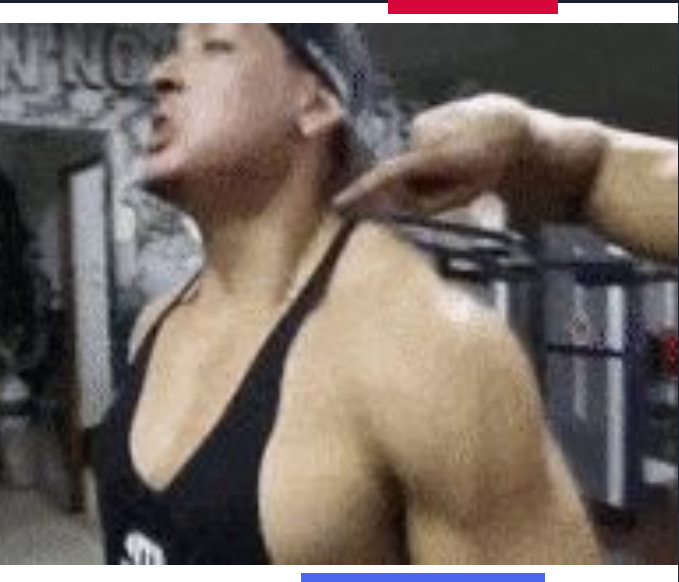
## PARTICIPANTES

- **Composite (pt 2):** sempre que a operação for solicitada, deve se executar todas as operações descendentes na hierarquia de forma recursiva
- **Ciente:** elemento que inicia a raiz da composição



## UTILIZAÇÃO

O cliente inicia uma raiz da composição, adicionando novas partes da **composição** (compostos) ou adicionando **operações terminais** (folhas). A operação executada a partir da raiz será o resultado da composição de toda a árvore de componentes registrados.

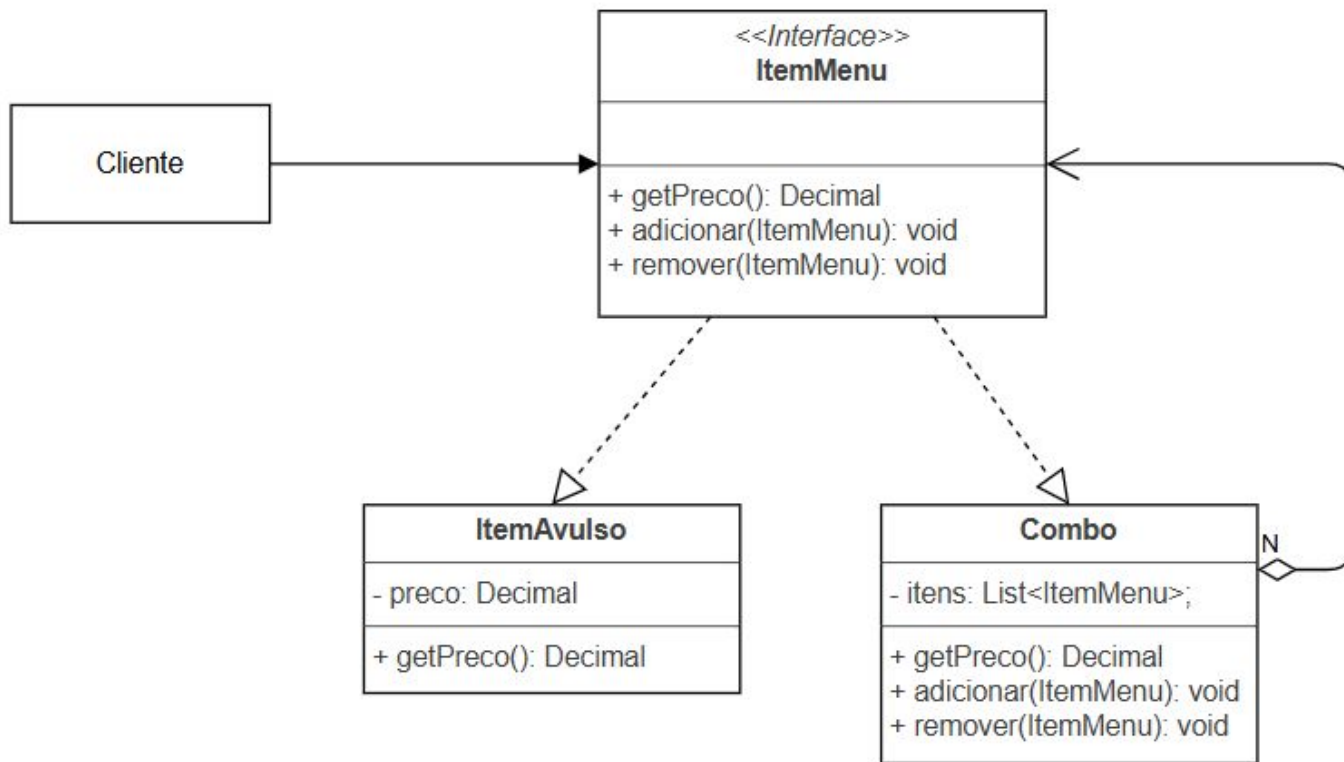


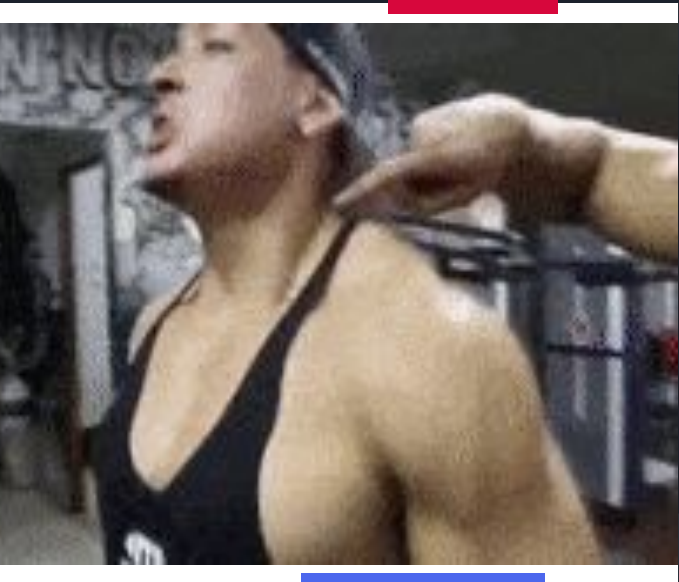
## HORA DE EXERCÍCIO

Imagine um sistema de pedidos para um restaurante, onde um cliente pode comprar itens “avulsos” ou “combos”. O atendente registra todos os itens do pedido e, ao final, ele deve ser capaz de calcular o valor final e informar ao cliente.

Vamos implementar este sistema.

# DIAGRAMA





## HORA DE EXERCÍCIO

Vamos implementar o sistema de arquivos do sistema operacional. Neste sistema, devemos ser capazes de listar o que temos em um diretório e ver o total de espaço ocupado por arquivos e diretórios.

**DIAGRAMA?**





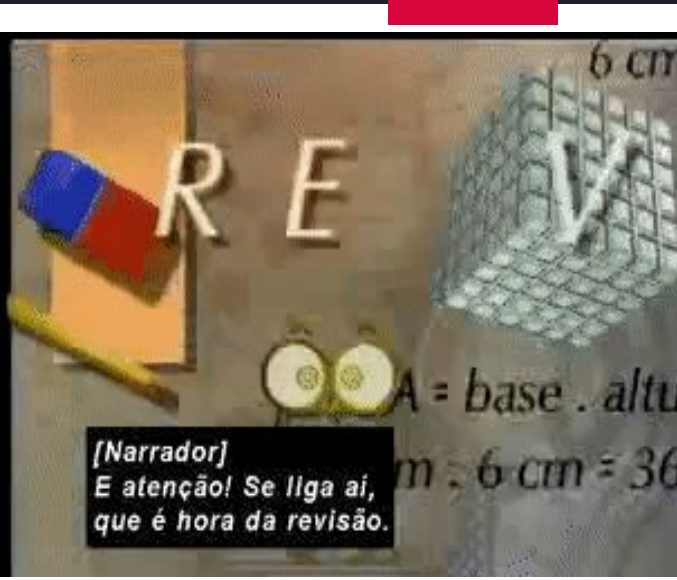
## APLICABILIDADE

- Precisamos representar no sistema uma hierarquia de objetos partes-todo
- Os clientes do sistema devem ser capazes de ignorar a diferença entre composições de objetos e objetos individuais, todos serão tratados da mesma forma uniforme



## CONSEQUÊNCIAS

- Hierarquia de classes com objetos primitivos e compostos
- Clientes simples, tratam os objetos da mesma forma
- Facilidade na adição de novas espécies de componentes
- Pode tornar o projeto genérico demais
- Inutilidade de métodos de composição em objetos individuais



## RECAPITULANDO

- Padrões de projeto estruturais
- Padrão Adapter
- Padrão Bridge
- Padrão Composite

# OBRIGADO!



**"ASKITIGUIRIDIDOUUUU!"**

**Xand Avião**